

CLAIMS REQUIRING RECONSTRUCTION

Which Claims to Re-Draft, and the Exact Wording to Use

*“A System and Method for Privacy-Preserving
Edge-Based Threat Detection in a Domestic Environment”*

Application reference	AR933 (Complete Specification, Form 2)
Claims to reconstruct	Claims 1, 5, and 7 only
Claims requiring no change	Claims 2, 3, 4, 6, 8, 9, 10
Inventors	Gonugondla Veera Manikanta; Dr Swathi Tangi
Prepared for	Adv. Pranav Bhat (IN/PA 4580), Patent Agent
Prepared by	Gonugondla Veera Manikanta (Inventor)
Source of record	Repository Manikanta25055/Garuda_26, branch main https://github.com/Manikanta25055/Garuda_26
Document status	Confidential — for prosecution use

This document suggests re-drafting **only** the claims that the working implementation does not support as presently worded. For each such claim it states **Why** the current wording is not enabled, **How** to fix it, and **What** exact replacement language could be used. All other claims are already aligned, or will be brought into alignment by code changes the inventor is making, and need no change. These are suggestions from the inventor’s side and are open to discussion.

1 How to Read This Document

These are the inventor’s suggestions, not instructions. The reconstructions proposed here reflect what the working code can honestly support today. They are offered to help align the claims with the implementation — they are not a direction to the agent. If any claim should be kept as originally drafted for strategy, scope, or prosecution reasons, please flag it and we will discuss before anything is changed. Where a divergence can be closed by a code change instead of a claim amendment, the inventor will make that change.

Three of the ten claims recite behaviour that the actual working code does not perform, and cannot be made to perform by drafting alone or by a small code change. Those three — **Claim 1**, **Claim 5**, and **Claim 7** — must be reconstructed. This document gives, for each:

- **Why** — the specific clause that is not enabled, and the technical reason the implementation cannot deliver it as worded.
- **How** — the drafting approach: what to re-scope, narrow, or replace.
- **What** — ready-to-insert replacement language, shown in a shaded box.

Every other claim is either already implemented exactly as written, or differs only in a parameter value that the inventor is correcting in code so the claim stays as drafted. Those claims need no change from our side. Section 2 lists the full disposition so the scope is clear.

2 Scope — Which Claims to Touch

Claim	Action	Reason
1	Reconstruct	Two clauses not enabled: the secondary network’s recited function, and the absolute restriction on video egress. See Sec. 3.
2	Keep (verify data)	Wording stands. The 50 fps figure depends on re-measurement — see the advisory in Sec. 4.
3	Keep	Code corrected to the 3-consecutive-frame gate; claim now matches.

Claim	Action	Reason
4	Keep	Code corrected to the 0.60 / 0.25–0.35 asymmetric thresholds; claim now matches.
5	Reconstruct	The “less than 4 milliseconds” figure is physically unattainable for the real secondary models. See Sec. 4.
6	Keep	Power-draw limit measured and satisfied.
7	Reconstruct	Independent method claim; mirrors Claim 1 and carries the same two defects. See Sec. 5.
8	Keep	Thermal limit satisfied.
9	Keep	20 ms hardware inference latency measured and satisfied.
10	Keep	Operating-mode flags under a single lock implemented as claimed.

One-line summary: we suggest re-drafting **Claim 1**, **Claim 5**, and **Claim 7** using the shaded “What” text below, and leaving every other claim untouched — subject to your review.

3 Claim 1 — System Claim (Reconstruct)

Claim 1 carries two clauses that the code does not support. They are independent of each other; both must be re-drafted.

3.1 Clause 1A — The Secondary Network’s Recited Function

3.1.1 Why (present wording is not enabled)

The claim recites the secondary network as one that re-checks the primary’s own class label:

“... at least one secondary verification neural network configured to independently re-evaluate a cropped image region ... and to generate a verification output confirming or rejecting a classification assigned by the primary detection neural network.”

The implemented secondary network does not do this. It is a binary *Safe-versus-Weapon* classifier (MobileNetV2) combined with a depth-variance anti-spoof (MiDaS Small), and it is run on *person* crops only. Weapon-class detections from the primary are treated as final outputs of the primary and are never routed to the secondary (`garuda_cascade.py:439--447`). The secondary therefore never “confirms or rejects” the primary’s weapon-class label; it answers a different question — “is this person associated with a weapon, and is the person a real three-dimensional subject rather than a photo or screen.” As worded, the clause describes a function the system does not perform and is not enabled for the weapon classes.

3.1.2 How (drafting approach)

Re-scope the secondary’s recited function to what it actually produces: a binary safe-versus-threat classification of the cropped region *plus* a depth-variance (anti-spoof) measure, with the alert gated on that combined output. Delete the language that implies re-verification of the primary’s own class label.

3.1.3 What (replacement language)

“... at least one secondary verification neural network 109 configured to receive a cropped image region corresponding to the detected region of interest and to generate a verification output, the verification output comprising (i) a binary safe-versus-threat classification of the cropped image region and (ii) a depth-variance measure indicative of whether the cropped image region corresponds to a three-dimensional physical subject or to a substantially planar surface, the host processor 103 being configured to gate the alert signal on the said verification output;”

3.2 Clause 1B — The Restriction on Video Egress

3.2.1 Why (present wording is not enabled)

The closing *wherein* of **Claim 1** recites an absolute bar on sending frames off the box:

*“... to restrict transmission of the sequence of video frames captured by the camera sensor to **any device external** to the host processor and the neural processing unit, thereby providing the privacy-preserving edge-based threat detection.”*

The system streams live MJPEG / WebRTC video to remote browsers (`Garuda_web.py:90,966--968`); raw frames are obscured only in *privacy* mode. In normal operation the frame sequence does leave the host, so an absolute “any device external” restriction is contradicted by a shipped feature and is not enabled.

3.2.2 How (drafting approach)

Narrow the restriction from “any external device” to the boundary that is truly held: frames and inference outputs never reach a *third-party* server; they stay within the user-owned trust boundary (evidence goes by SSH/SFTP to a user-owned store, no cloud). This is the honest and defensible scope. (If instead the applicant prefers the absolute wording, it can be kept only by a code change that forces the privacy blur on *all* streamed frames in every mode — the inventor can implement this; note it to the agent.)

3.2.3 What (replacement language)

“... and to restrict transmission of the sequence of video frames captured by the camera sensor 101, and of any inference output derived therefrom, to any server outside a user-owned trust boundary, such that neither the video frames nor the inference outputs are transmitted to a third-party server, thereby providing the privacy-preserving edge-based threat detection.”

4 Claim 5 — Secondary-Stage Latency (Reconstruct)

4.0.1 Why (present wording is not enabled)

Claim 5 recites:

“... executes the at least one secondary verification neural network within a combined latency of less than 4 milliseconds per cropped image region ...”

The two real secondary models measure MobileNetV2 ≈ 36 ms and MiDaS Small ≈ 29 ms per crop on the Hailo-8L (`garuda_cascade.py:451, 457`). A combined secondary latency under 4 ms is not attainable for these models on this accelerator — it is a model-size / hardware limit, not something a code change can reach. The figure is unsupported and must be re-drafted, not coded around.

4.0.2 How (drafting approach)

Two truthful options; pick one:

- **Option 1 — state the real number.** Replace “less than 4 milliseconds” with the actual measured combined secondary latency (of order 30–40 ms), re-measured with the live secondary in the loop, and soften the surrounding “real-time” wording accordingly.
- **Option 2 — claim the fast path.** Recite the lightweight RGB-variance anti-spoof proxy, which genuinely runs in under 1 ms without invoking the NPU (`garuda_cascade.py:88–90, 129–143`), as the real-time verification stage, and describe the MobileNet/MiDaS path as a higher-assurance mode. This keeps a truthful sub-millisecond figure.

4.0.3 What (replacement language)

Option 1 (measured value): “. . . executes the at least one secondary verification neural network 109 within a combined latency of not more than [MEASURED] milliseconds per cropped image region, the combined latency being measured independently of a frame processing rate of the primary detection neural network 107 . . . ”

Option 2 (fast-path proxy): “. . . executes, on the said at least a third one of the plurality of processing cores and without invoking the neural processing unit 105, a variance-based anti-spoof verification stage within a latency of less than 1 millisecond per cropped image region, the said verification stage rejecting a cropped image region whose pixel-intensity variance is below a spoof-rejection threshold . . . ”

Data note affecting Claims 2 and 5. In the production server the secondary worker is presently a placeholder that logs and returns without running a network (`Garuda_web.py: 445--478`). Any headline figure captured on that build — the ≈ 52 fps “under full concurrent load” (**Claim 2**) and the secondary-stage latency (**Claim 5**) — was recorded with the second network *not executing*. Before either number is relied on for prosecution, re-measure with a live secondary in the loop. This is a measurement action, not a drafting action; the **Claim 2** *wording* can stand if the number holds.

5 Claim 7 — Method Claim (Reconstruct)

Claim 7 is the independent method claim and mirrors **Claim 1**. It carries the same two defects, in step form, and must receive the same two reconstructions.

5.0.1 Why (present wording is not enabled)

Two steps repeat the **Claim 1** problems:

- “*independently re-evaluating . . . confirming or rejecting a classification assigned by the primary detection neural network*” — same issue as Clause 1A.
- “*restricting . . . transmission of the sequence of video frames . . . to any device external to the host processor and the neural processing unit*” — same issue as Clause 1B.

5.0.2 How (drafting approach)

Apply the identical re-scoping used for **Claim 1**, cast as method steps.

5.0.3 What (replacement language)

Verification step (replaces the “re-evaluating... confirming or rejecting” step):

“generating, by the at least one secondary verification neural network 109, a verification output from the cropped image region, the verification output comprising (i) a binary safe-versus-threat classification of the cropped image region and (ii) a depth-variance measure indicative of whether the cropped image region corresponds to a three-dimensional physical subject or to a substantially planar surface, and gating the alert signal on the said verification output;”

Egress step (replaces the “restricting... to any device external” step):

“restricting, by the host processor 103, transmission of the sequence of video frames captured by the camera sensor 101, and of any inference output derived therefrom, to any server outside a user-owned trust boundary, such that neither the video frames nor the inference outputs are transmitted to a third-party server, thereby providing the privacy-preserving edge-based threat detection.”

6 Closing

Our suggestion is to reconstruct **Claim 1**, **Claim 5**, and **Claim 7** using the shaded “What” text above, and to leave **Claim 2**, **Claim 3**, **Claim 4**, **Claim 6**, **Claim 8**, **Claim 9**, and **Claim 10** as drafted — the inventor is correcting the code so those claims remain true. The only non-drafting item is the **Claim 2/Claim 5** re-measurement flagged in the advisory above. These are proposals from the inventor’s side; if any claim should be preserved as originally drafted, we would like to discuss it with you first. All file-and-line references are against Manikanta25055/Garuda_26, branch main (https://github.com/Manikanta25055/Garuda_26), and can be verified independently.

Prepared by the inventor from a line-by-line source audit. Suggestions only — open to discussion.
Confidential.